CPU Scheduling

Student Assignment Worksheet – Solutions

Name	Register No.
Kaviya R	**192521075**

Class	Date
Operating System	**28 October 2026**

Instructions followed for every problem

- Draw a Gantt chart, including any CPU idle periods.
- Find each process's completion time (CT).
- Calculate each process's turnaround time (TAT) and waiting time (WT).
- Calculate the average turnaround time and average waiting time.

Formulas

Quantity	Formula
Turnaround time	TAT = CT − AT
Waiting time	WT = TAT − BT
Average TAT	Sum of TAT values ÷ number of processes
Average WT	Sum of WT values ÷ number of processes

(AT = arrival time, BT = burst time, CT = completion time – the clock time at which the process finishes.)

Assumptions and tie-breaking rules applied

- All times are in milliseconds (ms); one CPU, no I/O waiting, zero context-switch overhead.
- If processes arrive simultaneously, their listed order is used.
- For equal burst times, remaining times or priorities, the earlier-arriving process is preferred, then the listed order.

How the Gantt charts are drawn

Each coloured block is a period in which one process uses the CPU; the numbers under the chart are clock times (ms) at which the CPU switches. Grey hatched blocks marked IDLE are periods in which the CPU has no ready process. Every problem lists the scheduling decision at each step, then the Gantt chart, the CT / TAT / WT table with the arithmetic shown, the averages, a verification check and the answers to the additional questions.

Problem 1: FCFS — All Processes Arrive Together

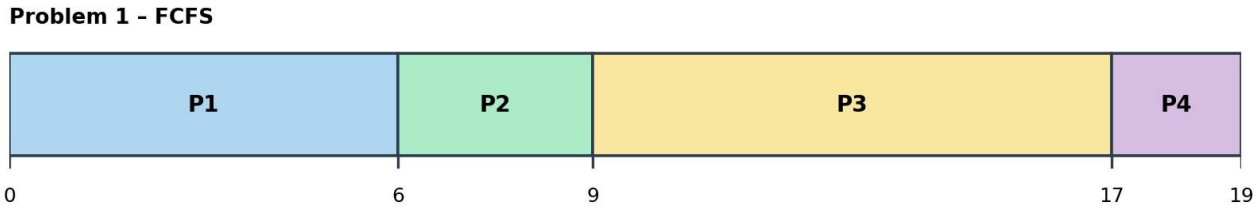
Process	Arrival time (ms)	Burst time (ms)
P1	0	6
P2	0	3
P3	0	8
P4	0	2

Rule applied

First Come, First Served (FCFS) is non-preemptive: the CPU is given to the process that arrived first and it runs to completion. Processes that arrive at the same time are served in their listed order.

Step-by-step solution

- **Step 1.** All four processes arrive at t = 0, so the tie-breaking rule (listed order) gives the execution order **P1 → P2 → P3 → P4**.
- **Step 2.** t = 0: P1 starts at once and runs for 6 ms → **CT(P1) = 0 + 6 = 6**.
- **Step 3.** t = 6: P2 starts and runs for 3 ms → **CT(P2) = 6 + 3 = 9**.
- **Step 4.** t = 9: P3 starts and runs for 8 ms → **CT(P3) = 9 + 8 = 17**.
- **Step 5.** t = 17: P4 starts and runs for 2 ms → **CT(P4) = 17 + 2 = 19**.
- **Step 6.** No process is waiting for an arrival, so the CPU is never idle (0 – 19 ms). **Gantt chart**



Text form: 0 |P1| 6 |P2| 9 |P3| 17 |P4| 19

Calculations

Process	AT	BT	CT (ms)	TAT = CT − AT (ms)	WT = TAT − BT (ms)
P1	0	6	6	6 − 0 = 6	6 − 6 = 0
P2	0	3	9	9 − 0 = 9	9 − 3 = 6
P3	0	8	17	17 − 0 = 17	17 − 8 = 9
P4	0	2	19	19 − 0 = 19	19 − 2 = 17

Average TAT = (6 + 9 + 17 + 19) ÷ 4 = 51 ÷ 4 = **12.75 ms**

Average WT = (0 + 6 + 9 + 17) ÷ 4 = 32 ÷ 4 = **8 ms**

Verification: total burst time = 6 + 3 + 8 + 2 = 19 ms = CPU busy time (19 ms); no idle time, so the last completion time is 19 ms. Also Σ WT = Σ TAT − Σ BT ⇒ 51 − 19 = 32 ✓.

Additional question

Which process has the longest waiting time? P4 (WT = 17 ms). P4 arrived at 0 but could only start at 17, after P1, P2 and P3 had used the CPU for 6 + 3 + 8 = 17 ms. Although P4 is the shortest job (2 ms), it waits the longest because FCFS ignores burst length – this is the **convoy effect**.

Problem 2: FCFS — Different Arrival Times and CPU Idle Time

Process	Arrival time (ms)	Burst time (ms)
P1	0	3
P2	5	4
P3	6	2
P4	12	3

Rule applied

FCFS serves processes strictly in order of arrival. If the ready queue is empty and no process has arrived yet, the CPU stays **idle** until the next arrival.

Step-by-step solution

- **Step 1.** t = 0: only P1 has arrived (P2, P3, P4 arrive at 5, 6, 12). P1 runs 0 – 3 → **CT(P1) = 3**.
- **Step 2.** t = 3: the ready queue is empty and the next arrival (P2) is at t = 5 → **CPU IDLE from 3 to 5** (2 ms).
- **Step 3.** t = 5: P2 arrives and starts immediately; runs 5 – 9 → **CT(P2) = 9**. P3 arrives at t = 6 while P2 is running and waits in the ready queue.
- **Step 4.** t = 9: P3 is the only waiting process → runs 9 – 11 → **CT(P3) = 11** (it waited 9 − 6 = 3 ms).
- **Step 5.** t = 11: the ready queue is empty again and P4 arrives at t = 12 → **CPU IDLE from 11 to 12** (1 ms).
- **Step 6.** t = 12: P4 starts immediately and runs 12 – 15 → **CT(P4) = 15**. **Gantt chart**



Text form: 0 |P1| 3 |IDLE| 5 |P2| 9 |P3| 11 |IDLE| 12 |P4| 15

Calculations

Process	AT	BT	CT (ms)	TAT = CT − AT (ms)	WT = TAT − BT (ms)
P1	0	3	3	3 − 0 = 3	3 − 3 = 0
P2	5	4	9	9 − 5 = 4	4 − 4 = 0
P3	6	2	11	11 − 6 = 5	5 − 2 = 3
P4	12	3	15	15 − 12 = 3	3 − 3 = 0

Average TAT = (3 + 4 + 5 + 3) ÷ 4 = 15 ÷ 4 = **3.75 ms**

Average WT = (0 + 0 + 3 + 0) ÷ 4 = 3 ÷ 4 = **0.75 ms**

Verification: total burst time = 3 + 4 + 2 + 3 = 12 ms = CPU busy time (12 ms); idle time = 3 ms; last completion = 15 ms. Also Σ WT = Σ TAT − Σ BT ⇒ 15 − 12 = 3 ✓.

Additional questions

Every CPU idle interval: 3 – 5 ms and **11 – 12 ms** (marked IDLE in the Gantt chart).

Total idle time between 0 and the completion of all processes (15 ms): (5 − 3) + (12 − 11) = 2 + 1 = **3 ms**. Cross-check: busy time = 3 + 4 + 2 + 3 = 12 ms; 15 − 12 = 3 ms ✓. CPU utilisation = 12 ÷ 15 = 80 %.

Problem 3: SJF — All Processes Arrive Together

Process	Arrival time (ms)	Burst time (ms)
P1	0	7
P2	0	4
P3	0	1
P4	0	5
P5	0	2

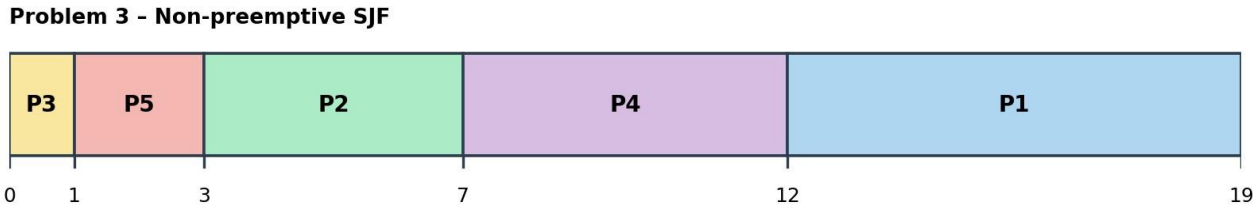
Rule applied

Non-preemptive Shortest Job First (SJF): whenever the CPU becomes free, choose the ready process with the smallest burst time; it then runs to completion.

Step-by-step solution

- **Step 1.** All five processes arrive at t = 0, so all are ready. Sort them by burst time: **P3 (1) < P5 (2) < P2 (4) < P4 (5) < P1 (7)** (no equal bursts, so no tie-breaking is needed).
- **Step 2.** t = 0: P3 runs 0 – 1 → CT = 1.
- **Step 3.** t = 1: P5 runs 1 – 3 → CT = 3.
- **Step 4.** t = 3: P2 runs 3 – 7 → CT = 7.
- **Step 5.** t = 7: P4 runs 7 – 12 → CT = 12.
- **Step 6.** t = 12: P1 runs 12 – 19 → CT = 19.

Gantt chart



Text form: 0 |P3| 1 |P5| 3 |P2| 7 |P4| 12 |P1| 19

Calculations (SJF)

Process	AT	BT	CT (ms)	TAT = CT − AT (ms)	WT = TAT − BT (ms)
P1	0	7	19	19 − 0 = 19	19 − 7 = 12
P2	0	4	7	7 − 0 = 7	7 − 4 = 3
P3	0	1	1	1 − 0 = 1	1 − 1 = 0
P4	0	5	12	12 − 0 = 12	12 − 5 = 7
P5	0	2	3	3 − 0 = 3	3 − 2 = 1

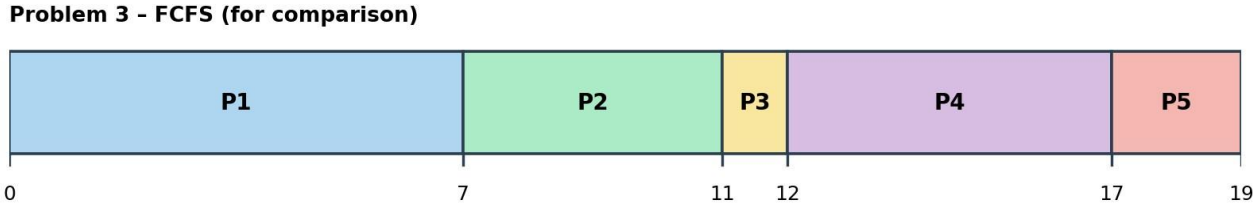
Average TAT = (19 + 7 + 1 + 12 + 3) ÷ 5 = 42 ÷ 5 = **8.4 ms**

Average WT = (12 + 3 + 0 + 7 + 1) ÷ 5 = 23 ÷ 5 = **4.6 ms**

Verification: total burst time = 7 + 4 + 1 + 5 + 2 = 19 ms = CPU busy time (19 ms); no idle time, so the last completion time is 19 ms. Also Σ WT = Σ TAT − Σ BT ⇒ 42 − 19 = 23 ✓.

Additional question – repeat using FCFS
With FCFS the processes run in listed order P1, P2, P3, P4, P5: **Gantt**

chart



Text form: 0 |P1| 7 |P2| 11 |P3| 12 |P4| 17 |P5| 19

Process	AT	BT	CT (ms)	TAT = CT − AT (ms)	WT = TAT − BT (ms)
P1	0	7	7	7 − 0 = 7	7 − 7 = 0
P2	0	4	11	11 − 0 = 11	11 − 4 = 7
P3	0	1	12	12 − 0 = 12	12 − 1 = 11
P4	0	5	17	17 − 0 = 17	17 − 5 = 12
P5	0	2	19	19 − 0 = 19	19 − 2 = 17

Average TAT = (7 + 11 + 12 + 17 + 19) ÷ 5 = 66 ÷ 5 = **13.2 ms**

Average WT = (0 + 7 + 11 + 12 + 17) ÷ 5 = 47 ÷ 5 = **9.4 ms**

Algorithm	Average TAT (ms)	Average WT (ms)
FCFS	13.2	9.4
Non-preemptive SJF	8.4	4.6

Answer: SJF gives the lower average waiting time – 4.6 ms against 9.4 ms for FCFS (a reduction of 4.8 ms, about 51 %). SJF lets the short jobs (P3, P5) finish first, so fewer processes wait behind the long job P1; FCFS made all other processes wait 7 ms behind P1 first. SJF is also better for average turnaround time (8.4 ms vs 13.2 ms).

Problem 4: SJF — Different Arrival Times

Process	Arrival time (ms)	Burst time (ms)
P1	0	8

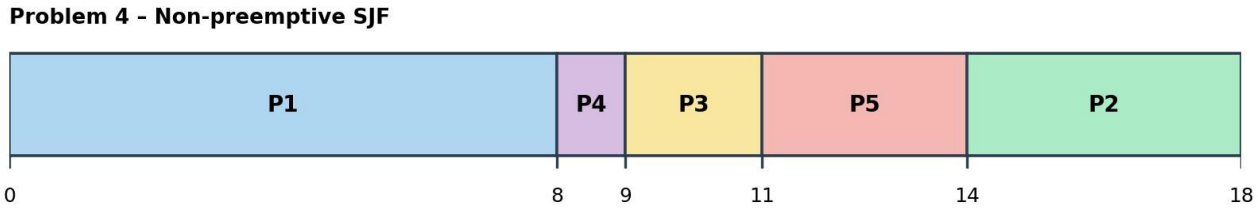
P2	1	4
P3	2	2
P4	3	1
P5	10	3

Rule applied

Non-preemptive SJF: at every scheduling point, only processes that **have already arrived** are considered; the one with the smallest burst time is chosen and runs to completion. **Step-by-step solution (decision at every scheduling point)**

Time	Ready processes (burst)	Chosen	Reason / execution
0	P1	P1 (8)	P1 – the only arrived process; runs 0 – 8 without interruption
8	P2 (4), P3 (2), P4 (1)	P4 (burst 1)	P4 is the shortest → runs 8 – 9
9	P2 (4), P3 (2)	P3 (burst 2)	P3 runs 9 – 11
11	P2 (4), P5 (3) [P5 arrived at 10]	P5 (burst 3)	P5 runs 11 – 14
14	P2 (4)	P2	P2 runs 14 – 18

While P1 runs, P2 (t = 1), P3 (t = 2) and P4 (t = 3) arrive and wait; P5 arrives at t = 10 (while P3 is running), so it is also ready and competes with P2 at t = 11. **Gantt chart**



Text form: 0 |P1| 8 |P4| 9 |P3| 11 |P5| 14 |P2| 18

Calculations

Process	AT	BT	CT (ms)	TAT = CT − AT (ms)	WT = TAT − BT (ms)
P1	0	8	8	8 − 0 = 8	8 − 8 = 0
P2	1	4	18	18 − 1 = 17	17 − 4 = 13
P3	2	2	11	11 − 2 = 9	9 − 2 = 7
P4	3	1	9	9 − 3 = 6	6 − 1 = 5
P5	10	3	14	14 − 10 = 4	4 − 3 = 1

Average TAT = (8 + 17 + 9 + 6 + 4) ÷ 5 = 44 ÷ 5 = **8.8 ms**

Average WT = (0 + 13 + 7 + 5 + 1) ÷ 5 = 26 ÷ 5 = **5.2 ms**

Verification: total burst time = 8 + 4 + 2 + 1 + 3 = 18 ms = CPU busy time (18 ms); no idle time, so the last completion time is 18 ms. Also Σ WT = Σ TAT − Σ BT ⇒ 44 − 18 = 26 ✓.

Additional questions

Which process runs immediately after P1? P4. At t = 8 the ready processes are P2 (4 ms), P3 (2 ms) and P4 (1 ms); P4 has the shortest burst, even though P2 arrived earlier.

Why does P4 not interrupt P1 when it arrives (t = 3)? Because this is **non-preemptive** SJF: once the CPU is given to P1 it keeps it until P1 finishes, even though P4's burst (1 ms) is smaller than P1's remaining 5 ms. (Only a preemptive version, SRTF, would switch.)

Problem 5: SRTF — Preemptive Scheduling

Process	Arrival time (ms)	Burst time (ms)
P1	0	9
P2	1	5
P3	2	2
P4	4	1

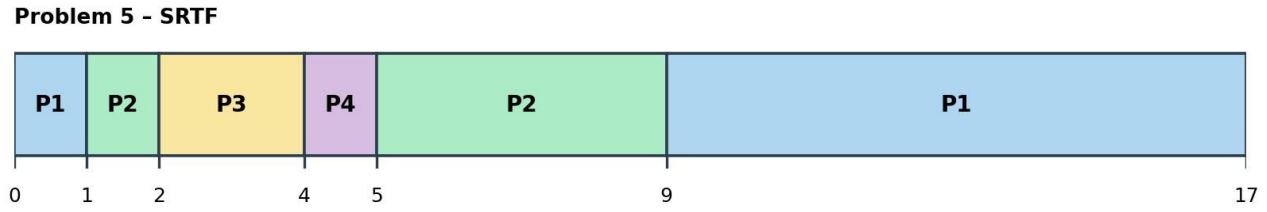
Rule applied

Shortest Remaining Time First (SRTF) is the preemptive version of SJF. Whenever a process arrives or one finishes, the process with the smallest **remaining** time runs. A running process is preempted only if a new arrival has a **strictly smaller** remaining time.

Step-by-step solution (event trace)

Time	Event	Remaining times of ready processes	Decision
0	P1 arrives	P1 (9)	Run P1
1	P2 arrives	P1 (8), P2 (5)	5 < 8 → **preempt P1**; run P2
2	P3 arrives	P1 (8), P2 (4), P3 (2)	2 < 4 → **preempt P2**; run P3
4	P3 completes; P4 arrives	P1 (8), P2 (4), P4 (1)	Smallest remaining = 1 (P4) → run P4
5	P4 completes	P1 (8), P2 (4)	4 < 8 → run P2 (resumes)
9	P2 completes	P1 (8)	Run P1 (resumes)
17	P1 completes	—	All processes finished

Gantt chart



Text form: 0 |P1| 1 |P2| 2 |P3| 4 |P4| 5 |P2| 9 |P1| 17

Calculations

Process	AT	BT	CT (ms)	TAT = CT − AT (ms)	WT = TAT − BT (ms)
P1	0	9	17	17 − 0 = 17	17 − 9 = 8
P2	1	5	9	9 − 1 = 8	8 − 5 = 3
P3	2	2	4	4 − 2 = 2	2 − 2 = 0

P4	4	1	5	5 − 4 = 1	1 − 1 = 0

Average TAT = (17 + 8 + 2 + 1) ÷ 4 = 28 ÷ 4 = **7 ms**

Average WT = (8 + 3 + 0 + 0) ÷ 4 = 11 ÷ 4 = **2.75 ms**

Verification: total burst time = 9 + 5 + 2 + 1 = 17 ms = CPU busy time (17 ms); no idle time, so the last completion time is 17 ms. Also Σ WT = Σ TAT − Σ BT ⇒ 28 − 17 = 11 ✓.

Additional questions

Every preemption: (1) at **t = 1** P1 (remaining 8) is interrupted by P2 (remaining 5); (2) at **t = 2** P2 (remaining 4) is interrupted by P3 (remaining 2). The process at t = 4 is not a preemption – P3 had already completed.

At time 4, which process runs next, and why? P4. P3 has just completed. The ready processes are P1 (remaining 8), P2 (remaining 4) and P4 (remaining 1). SRTF selects the smallest remaining time, which is P4 with 1 ms.

Problem 6: Round Robin — Time Quantum of 2 ms

Process	Arrival time (ms)	Burst time (ms)
P1	0	5
P2	0	4
P3	0	2
P4	0	1

Rule applied

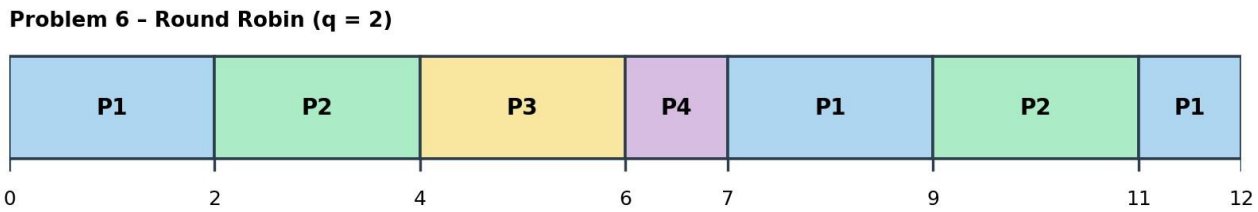
Round Robin (RR), q = 2 ms: processes wait in a FIFO ready queue; each runs for at most 2 ms. If it is unfinished it goes to the back of the queue. Arrivals at the exact end of a quantum are queued before the unfinished process is requeued (not needed here because all processes arrive at 0).

Step-by-step solution (slice by slice)

Ready queue at time 0 (before the first slice): **[P1, P2, P3, P4]**.

Slice	Interval (ms)	Runs	Ran for (ms)	Remaining after	Ready queue immediately after the slice
1	0 – 2	P1	2	3	[P2, P3, P4, P1]
2	2 – 4	P2	2	2	[P3, P4, P1, P2]
3	4 – 6	P3	2	0	[P4, P1, P2]
4	6 – 7	P4	1	0	[P1, P2]
5	7 – 9	P1	2	1	[P2, P1]
6	9 – 11	P2	2	0	[P1]
7	11 – 12	P1	1	0	[ ] (empty)

Remaining burst times: P1 5 → 3 → 1 → 0; P2 4 → 2 → 0; P3 2 → 0; P4 1 → 0. **Gantt chart**



Text form: 0 |P1| 2 |P2| 4 |P3| 6 |P4| 7 |P1| 9 |P2| 11 |P1| 12

Calculations

Process	AT	BT	CT (ms)	TAT = CT − AT (ms)	WT = TAT − BT (ms)
P1	0	5	12	12 − 0 = 12	12 − 5 = 7

P2	0	4	11	11 − 0 = 11	11 − 4 = 7
P3	0	2	6	6 − 0 = 6	6 − 2 = 4
P4	0	1	7	7 − 0 = 7	7 − 1 = 6

Average TAT = (12 + 11 + 6 + 7) ÷ 4 = 36 ÷ 4 = **9 ms**

Average WT = (7 + 7 + 4 + 6) ÷ 4 = 24 ÷ 4 = **6 ms**

Verification: total burst time = 5 + 4 + 2 + 1 = 12 ms = CPU busy time (12 ms); no idle time, so the last completion time is 12 ms. Also Σ WT = Σ TAT − Σ BT ⇒ 36 − 12 = 24 ✓.

Additional questions

Ready-queue order immediately after each time slice: after 0–2: [P2, P3, P4, P1]; after 2–4: [P3, P4, P1, P2]; after 4–6: [P4, P1, P2]; after 6–7: [P1, P2]; after 7–9: [P2, P1]; after 9–11: [P1]; after 11–12: [ ].

Which process finishes first? P3 – it completes at t = 6 ms (it needs exactly one quantum and gets it in the third slice). P4 finishes next at t = 7.

Problem 7: Round Robin — Processes Arrive During Execution

Process	Arrival time (ms)	Burst time (ms)
P1	0	7
P2	1	4
P3	3	2
P4	5	5

Rule applied

Round Robin with **q = 3 ms**. A process that arrives while another is running joins the back of the ready queue at its arrival time. If an arrival coincides with the end of a quantum, **the arrival is queued before the unfinished running process is requeued** (boundary-arrival rule).

Step-by-step solution (slice by slice)

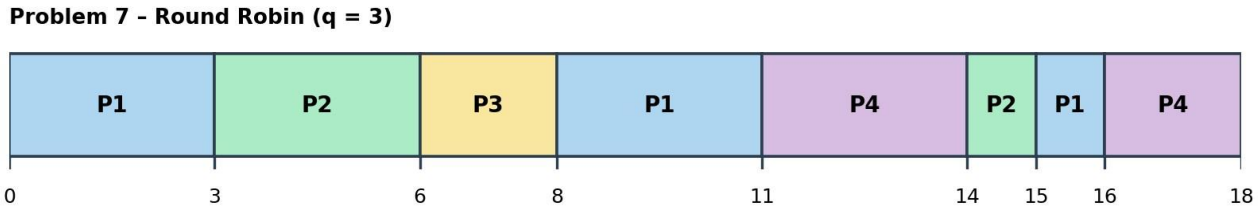
Ready queue at time 0 (before the first slice): **[P1]**.

Slice	Interval (ms)	Runs	Ran for (ms)	Remaining after	Ready queue immediately after the slice
1	0 – 3	P1	3	4	[P2, P3, P1]
2	3 – 6	P2	3	1	[P3, P1, P4, P2]
3	6 – 8	P3	2	0	[P1, P4, P2]
4	8 – 11	P1	3	1	[P4, P2, P1]
5	11 – 14	P4	3	2	[P2, P1, P4]
6	14 – 15	P2	1	0	[P1, P4]
7	15 – 16	P1	1	0	[P4]
8	16 – 18	P4	2	0	[ ] (empty)

Key queue moments:

- t = 1: P2 arrives and joins the queue behind the running P1 → queue [P2].
- t = 3: P1's quantum ends **and** P3 arrives at exactly t = 3. Boundary rule: P3 enters first, then P1 (4 ms left) is requeued → queue [P2, P3, P1].
- t = 5: P4 arrives while P2 is running (queue [P3, P1, P4]); at t = 6 P2 (1 ms left) is requeued behind them → [P3, P1, P4, P2].

Gantt chart



Text form: 0 |P1| 3 |P2| 6 |P3| 8 |P1| 11 |P4| 14 |P2| 15 |P1| 16 |P4| 18

Calculations

Process	AT	BT	CT (ms)	TAT = CT − AT (ms)	WT = TAT − BT (ms)
P1	0	7	16	16 − 0 = 16	16 − 7 = 9
P2	1	4	15	15 − 1 = 14	14 − 4 = 10
P3	3	2	8	8 − 3 = 5	5 − 2 = 3
P4	5	5	18	18 − 5 = 13	13 − 5 = 8

Average TAT = (16 + 14 + 5 + 13) ÷ 4 = 48 ÷ 4 = **12 ms**

Average WT = (9 + 10 + 3 + 8) ÷ 4 = 30 ÷ 4 = **7.5 ms**

Verification: total burst time = 7 + 4 + 2 + 5 = 18 ms = CPU busy time (18 ms); no idle time, so the last completion time is 18 ms. Also Σ WT = Σ TAT − Σ BT ⇒ 48 − 18 = 30 ✓.

Additional questions

Ready queue immediately after P1's first time slice (t = 3): [P2, P3, P1] (front → back).

Where is P3 placed when it arrives at t = 3? P3 arrives at exactly the moment P1's quantum expires. By the boundary-arrival rule, the newly arrived process enters the queue **before** the pre-empted process is put back. So P3 is placed behind P2 and **ahead of P1**; P1 goes to the very back.

Problem 8: Non-Preemptive Priority Scheduling

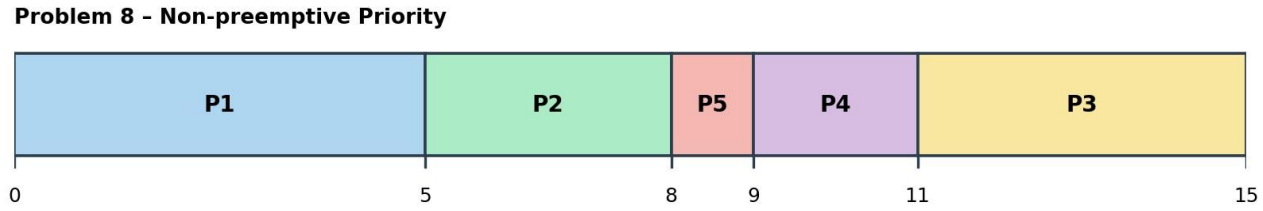
Process	Arrival time (ms)	Burst time (ms)	Priority
P1	0	5	3
P2	1	3	1
P3	2	4	4
P4	3	2	2
P5	4	1	1

Rule applied

Smaller number = higher priority. When the CPU becomes free, the **highest-priority ready process** is chosen; ties go to the earlier arrival, then listed order. A running process is never interrupted. **Step-by-step solution (decision at every scheduling point)**

Time	Ready processes (priority)	Chosen	Reason / execution
0	P1 (priority 3)	P1	Only arrived process; runs 0 – 5 (cannot be preempted)
5	P2 (1), P3 (4), P4 (2), P5 (1)	P2	P2 and P5 tie at priority 1 → earlier arrival (P2 at 1, P5 at 4) wins; runs 5 – 8
8	P3 (4), P4 (2), P5 (1)	P5	Highest priority (1); runs 8 – 9
9	P3 (4), P4 (2)	P4	Priority 2 beats 4; runs 9 – 11
11	P3 (4)	P3	Last process; runs 11 – 15

Gantt chart



Text form: 0 |P1| 5 |P2| 8 |P5| 9 |P4| 11 |P3| 15

Calculations

Process	AT	BT	CT (ms)	TAT = CT − AT (ms)	WT = TAT − BT (ms)
P1	0	5	5	5 − 0 = 5	5 − 5 = 0
P2	1	3	8	8 − 1 = 7	7 − 3 = 4
P3	2	4	15	15 − 2 = 13	13 − 4 = 9
P4	3	2	11	11 − 3 = 8	8 − 2 = 6
P5	4	1	9	9 − 4 = 5	5 − 1 = 4

Average TAT = (5 + 7 + 13 + 8 + 5) ÷ 5 = 38 ÷ 5 = **7.6 ms**

Average WT = (0 + 4 + 9 + 6 + 4) ÷ 5 = 23 ÷ 5 = **4.6 ms**

Verification: total burst time = 5 + 3 + 4 + 2 + 1 = 15 ms = CPU busy time (15 ms); no idle time, so the last completion time is 15 ms. Also Σ WT = Σ TAT − Σ BT ⇒ 38 − 15 = 23 ✓.

Additional questions

Why does P1 continue running after P2 arrives? P2 (priority 1) is more important than P1 (priority 3), but this is **non-preemptive** priority scheduling: a process that has started keeps the CPU until it finishes. P2 must wait until t = 5.

How is the tie between P2 and P5 resolved when the CPU becomes free? At t = 5 both have priority 1. The tie-breaking rule prefers the **earlier-arriving process**: P2 arrived at t = 1, P5 at t = 4, so P2 runs first (5 – 8) and P5 runs next (8 – 9).

Problem 9: Preemptive Priority Scheduling

Process	Arrival time (ms)	Burst time (ms)	Priority
P1	0	8	4
P2	1	4	2
P3	2	2	1
P4	3	3	3
P5	5	1	1

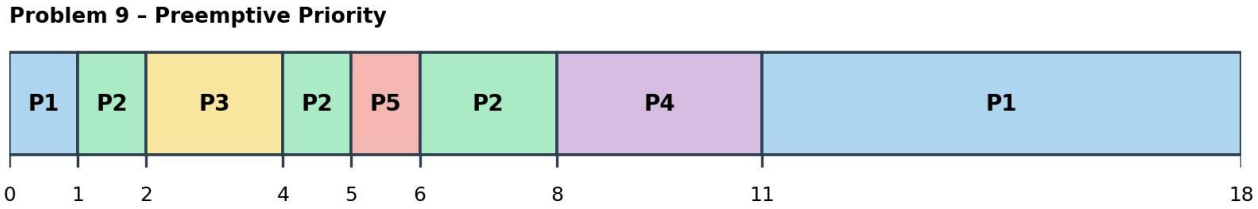
Rule applied

Smaller number = higher priority. At every arrival or completion the ready process with the **highest priority** (smallest number) runs; a running process is preempted when a newly arrived process has a **strictly higher** priority.

Step-by-step solution (event trace)

Time	Event	Ready processes (priority, remaining)	Decision
0	P1 arrives	P1 (p4)	Run P1
1	P2 arrives (p2)	P1 (p4, rem 7), P2 (p2, rem 4)	2 < 4 → **preempt P1**; run P2
2	P3 arrives (p1)	P1 (rem 7), P2 (p2, rem 3), P3 (p1, rem 2)	1 < 2 → **preempt P2**; run P3
3	P4 arrives (p3)	P3 (running), P1, P2, P4	P3 (p1) is still highest → continue P3
4	P3 completes	P1 (p4, rem 7), P2 (p2, rem 3), P4 (p3, rem 3)	Highest priority = P2 → resume P2
5	P5 arrives (p1)	P1, P2 (p2, rem 2), P4, P5 (p1, rem 1)	1 < 2 → **preempt P2**; run P5
6	P5 completes	P1 (p4, rem 7), P2 (p2, rem 2), P4 (p3, rem 3)	Resume P2 (6 – 8)
8	P2 completes	P1 (p4, rem 7), P4 (p3, rem 3)	3 < 4 → run P4 (8 – 11)
11	P4 completes	P1 (rem 7)	Resume P1 (11 – 18)

Gantt chart



Text form: 0 |P1| 1 |P2| 2 |P3| 4 |P2| 5 |P5| 6 |P2| 8 |P4| 11 |P1| 18

Calculations

Process	AT	BT	CT (ms)	TAT = CT − AT (ms)	WT = TAT − BT (ms)
P1	0	8	18	18 − 0 = 18	18 − 8 = 10
P2	1	4	8	8 − 1 = 7	7 − 4 = 3
P3	2	2	4	4 − 2 = 2	2 − 2 = 0
P4	3	3	11	11 − 3 = 8	8 − 3 = 5
Process	**AT**	**BT**	**CT (ms)**	**TAT = CT − AT (ms)**	**WT = TAT − BT (ms)**
P5	5	1	6	6 − 5 = 1	1 − 1 = 0

Average TAT = (18 + 7 + 2 + 8 + 1) ÷ 5 = 36 ÷ 5 = **7.2 ms**

Average WT = (10 + 3 + 0 + 5 + 0) ÷ 5 = 18 ÷ 5 = **3.6 ms**

Verification: total burst time = 8 + 4 + 2 + 3 + 1 = 18 ms = CPU busy time (18 ms); no idle time, so the last completion time is 18 ms. Also Σ WT = Σ TAT − Σ BT ⇒ 36 − 18 = 18 ✓.

Additional questions

Every preemption: (1) **t = 1** – P1 is preempted by P2 (priority 2 over 4); (2) **t = 2** – P2 is preempted by P3 (priority 1 over 2); (3) **t = 5** – P2 is preempted by P5 (priority 1 over 2). At t = 3 P4 arrives but does not preempt P3 (priority 3 is lower than 1).

Which process has the longest waiting time? P1 – WT = 10 ms (18 − 0 − 8). P1 has the lowest priority (4), so it runs only 1 ms at the start and then waits until all higher-priority work is done at t = 11.

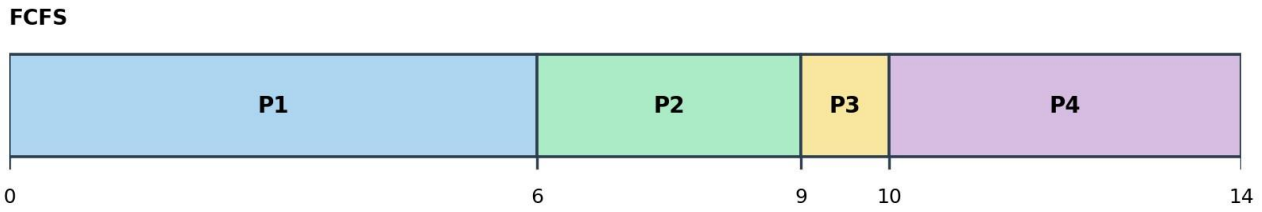
Problem 10: Compare Four Scheduling Algorithms

Process	Arrival time (ms)	Burst time (ms)
P1	0	6
P2	1	3
P3	2	1
P4	3	4

10.1 FCFS

- **Step 1.** Arrival order is P1 (0), P2 (1), P3 (2), P4 (3), so the execution order is P1 → P2 → P3 → P4.
- **Step 2.** P1: 0 – 6 (CT 6); P2: 6 – 9 (CT 9); P3: 9 – 10 (CT 10); P4: 10 – 14 (CT 14). No idle time.

Gantt chart



Text form: 0 |P1| 6 |P2| 9 |P3| 10 |P4| 14

Process	AT	BT	CT (ms)	TAT = CT − AT (ms)	WT = TAT − BT (ms)
P1	0	6	6	6 − 0 = 6	6 − 6 = 0
P2	1	3	9	9 − 1 = 8	8 − 3 = 5
P3	2	1	10	10 − 2 = 8	8 − 1 = 7
P4	3	4	14	14 − 3 = 11	11 − 4 = 7

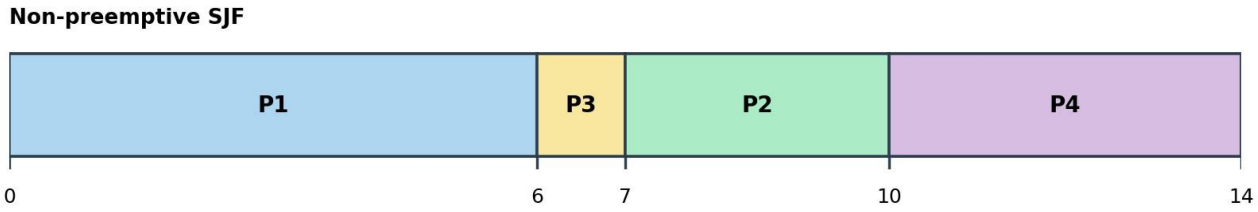
Average TAT = (6 + 8 + 8 + 11) ÷ 4 = 33 ÷ 4 = **8.25 ms**

Average WT = (0 + 5 + 7 + 7) ÷ 4 = 19 ÷ 4 = **4.75 ms**

10.2 Non-preemptive SJF

- **Step 1.** t = 0: only P1 has arrived → P1 runs 0 – 6 (cannot be preempted).
- **Step 2.** t = 6: ready processes are P2 (3), P3 (1), P4 (4) → shortest is **P3** → runs 6 – 7.
- **Step 3.** t = 7: ready P2 (3), P4 (4) → **P2** runs 7 – 10.
- **Step 4.** t = 10: only P4 remains → runs 10 – 14.

Gantt chart



Text form: 0 |P1| 6 |P3| 7 |P2| 10 |P4| 14

Process	AT	BT	CT (ms)	TAT = CT − AT (ms)	WT = TAT − BT (ms)
P1	0	6	6	6 − 0 = 6	6 − 6 = 0
P2	1	3	10	10 − 1 = 9	9 − 3 = 6
P3	2	1	7	7 − 2 = 5	5 − 1 = 4
P4	3	4	14	14 − 3 = 11	11 − 4 = 7

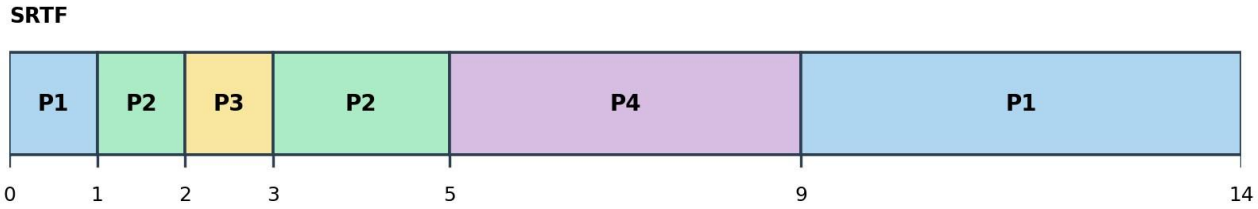
Average TAT = (6 + 9 + 5 + 11) ÷ 4 = 31 ÷ 4 = **7.75 ms**

Average WT = (0 + 6 + 4 + 7) ÷ 4 = 17 ÷ 4 = **4.25 ms**

10.3 SRTF (preemptive SJF)

Time	Event	Remaining times	Decision
0	P1 arrives	P1 (6)	Run P1
1	P2 arrives	P1 (5), P2 (3)	3 < 5 → preempt P1; run P2
2	P3 arrives	P1 (5), P2 (2), P3 (1)	1 < 2 → preempt P2; run P3
3	P3 completes; P4 arrives	P1 (5), P2 (2), P4 (4)	Smallest = P2 (2) → run P2 (3 – 5)
5	P2 completes	P1 (5), P4 (4)	4 < 5 → run P4 (5 – 9)
9	P4 completes	P1 (5)	Run P1 (9 – 14)

Gantt chart



Text form: 0 |P1| 1 |P2| 2 |P3| 3 |P2| 5 |P4| 9 |P1| 14

Process	AT	BT	CT (ms)	TAT = CT − AT (ms)	WT = TAT − BT (ms)
P1	0	6	14	14 − 0 = 14	14 − 6 = 8
P2	1	3	5	5 − 1 = 4	4 − 3 = 1

P3	2	1	3	3 − 2 = 1	1 − 1 = 0
P4	3	4	9	9 − 3 = 6	6 − 4 = 2

Average TAT = (14 + 4 + 1 + 6) ÷ 4 = 25 ÷ 4 = **6.25 ms**

Average WT = (8 + 1 + 0 + 2) ÷ 4 = 11 ÷ 4 = **2.75 ms**

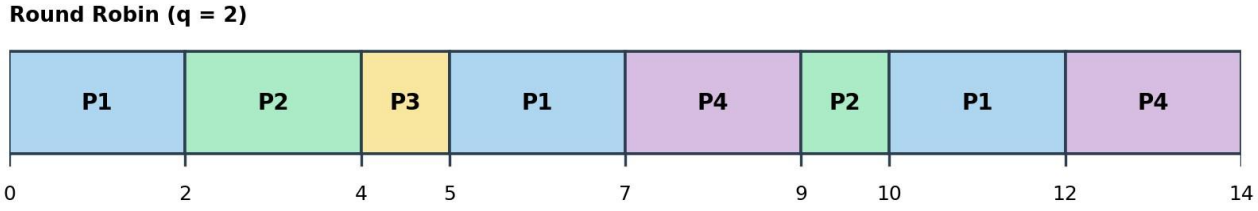
10.4 Round Robin (q = 2 ms)

Arrivals at exact slice boundaries (P3 at t = 2) are queued before the unfinished process is requeued.

Ready queue at time 0 (before the first slice): **[P1]**.

Slice	Interval (ms)	Runs	Ran for (ms)	Remaining after	Ready queue immediately after the slice
1	0 – 2	P1	2	4	[P2, P3, P1]
2	2 – 4	P2	2	1	[P3, P1, P4, P2]
3	4 – 5	P3	1	0	[P1, P4, P2]
4	5 – 7	P1	2	2	[P4, P2, P1]
5	7 – 9	P4	2	2	[P2, P1, P4]
6	9 – 10	P2	1	0	[P1, P4]
7	10 – 12	P1	2	0	[P4]
8	12 – 14	P4	2	0	[ ] (empty)

Gantt chart



Text form: 0 |P1| 2 |P2| 4 |P3| 5 |P1| 7 |P4| 9 |P2| 10 |P1| 12 |P4| 14

Process	AT	BT	CT (ms)	TAT = CT − AT (ms)	WT = TAT − BT (ms)
P1	0	6	12	12 − 0 = 12	12 − 6 = 6
P2	1	3	10	10 − 1 = 9	9 − 3 = 6
P3	2	1	5	5 − 2 = 3	3 − 1 = 2
P4	3	4	14	14 − 3 = 11	11 − 4 = 7

Average TAT = (12 + 9 + 3 + 11) ÷ 4 = 35 ÷ 4 = **8.75 ms**

Average WT = (6 + 6 + 2 + 7) ÷ 4 = 21 ÷ 4 = **5.25 ms**

10.5 Comparison table

Algorithm	Average TAT (ms)	Average WT (ms)
FCFS	8.25	4.75
Non-preemptive SJF	7.75	4.25
SRTF	6.25	2.75
Algorithm	**Average TAT (ms)**	**Average WT (ms)**
Round Robin (q = 2 ms)	8.75	5.25

Execution order of each algorithm

Algorithm	Order of CPU slices
FCFS	P1 → P2 → P3 → P4
Non-preemptive SJF	P1 → P3 → P2 → P4
SRTF	P1 → P2 → P3 → P2 → P4 → P1
Round Robin (q = 2)	P1 → P2 → P3 → P1 → P4 → P2 → P1 → P4

Answers to the additional questions

1. **Lowest average waiting time: SRTF – 2.75 ms** (SJF 4.25, FCFS 4.75, RR 5.25). SRTF always runs the process with the least remaining work, so short jobs P3 and P2 are not stuck behind P1.

2. **Does it also give the lowest average turnaround time? Yes.** SRTF has the lowest average TAT, 6.25 ms (SJF 7.75, FCFS 8.25, RR 8.75). Because Σ TAT = Σ WT + Σ BT and the total burst time (14 ms) is the same for every algorithm, the algorithm with the lowest average WT must also have the lowest average TAT.

3. **Do any algorithms produce the same execution order? No** – all four Gantt charts are different. FCFS (P1, P2, P3, P4) and non-preemptive SJF (P1, P3, P2, P4) are the closest: both run P1 first and P4 last and differ only in the order of P2 and P3. SRTF and RR contain preempted, split slices of the processes, so their orders are different from each other and from FCFS/SJF. (P4 happens to have WT
= 7 ms in FCFS, SJF and RR – the same waiting time, but not the same order.)

Summary of Results

Problem	Algorithm	Average TAT (ms)	Average WT (ms)
Problem 1	FCFS	12.75	8
Problem 2	FCFS (with idle)	3.75	0.75
Problem 3	SJF (non-preemptive)	8.4	4.6

Problem 4	SJF (non-preemptive)	8.8	5.2
Problem 5	SRTF	7	2.75
Problem 6	Round Robin q = 2	9	6
Problem 7	Round Robin q = 3	12	7.5
Problem 8	Priority (non-preemptive)	7.6	4.6
Problem 9	Priority (preemptive)	7.2	3.6
Problem 10	FCFS	8.25	4.75
Problem 10	SJF	7.75	4.25
Problem 10	SRTF	6.25	2.75
Problem 10	RR q = 2	8.75	5.25